Technical Support for Rural Startups

A Community Engagement Project Report

Submitted by

Mr. Mergu Pravin Madhukar
Roll No. 85
Mr. Maurya Chandar Ravindrakumar
Roll No. 84
Mr. Malik Gulma Gaus
Roll No. 82
Mr. Machha Yogesh Kamlakar
Roll No. 81

in partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE
in
INFORMATION TECHNOLOGY

under the guidance of

Mrs. Hera Kausar
Department of Information Technology

Padmashri Annasaheb Jadhav Bharatiya Samaj Unnati Mandal's
B.N.N. College (Arts, Science & Commerce), Bhiwandi.

Sem V

A.Y. 2026-2027

Padmashri Annasaheb Jadhav Bharatiya Samaj Unnati Mandal's
B.N.N. College (Arts, Science & Commerce), Bhiwandi.
(Self-Funded Courses)
Department of Information Technology
A.Y. 2026-2027

Certificate

This is to certify that **Mr. Mergu Pravin Madhukar**, **Mr. Maurya Chandar Ravindrakumar**, **Mr. Malik Gulma Gaus** and **Mr. Machha Yogesh Kamlakar** of T.Y.B.Sc.IT (Sem V) class have satisfactorily completed the Community Engagement Project on **Technical Support for Rural Startups**, to be submitted in the partial fulfillment for the award of Bachelor of Science in Information Technology during the academic year **2026 - 2027**.

Date of Submission:_____

Project Guide
(Mrs. Hera Kausar)

Head/Incharge,
Department of Information Technology

College Seal

Signature of Examiner

DECLARATION

We hereby declare that the Community Engagement Project report titled “Technical Support for Rural Startups” has been prepared by us as part of our academic work for T.Y.B.Sc. Information Technology, Semester V.

The project combines field visits to rural micro-businesses near Bhiwandi with the development of practical software tools. The report documents the visits, the problems observed, the solutions developed and the outcomes.

We confirm that the work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

Signature of the Students

Mr. Mergu Pravin Madhukar

Mr. Maurya Chandar Ravindrakumar

Mr. Malik Gulma Gaus

Mr. Machha Yogesh Kamlakar

Place:- Bhiwandi

Date:-_____

ABSTRACT

The Community Engagement Project “Technical Support for Rural Startups” aimed to help small village businesses near Bhiwandi, Maharashtra, adopt simple digital tools. Many such enterprises depend on manual records and repetitive work, and they rarely have access to patient, hands-on technical guidance.

Four enterprises were visited: Srividyaniketan Classes in Bhiwandi, Baba ID Cards Solutions in Shelar, V Tech ID Solutions in Kariwali and Digital ID Solutions in Khoni Gaon. During each visit the team observed the daily work, identified the main difficulty and set up a suitable solution together with the owner.

Based on the problems observed, a class management web application and three photo-processing tools were developed: an automatic face and shoulder cropping tool, a bulk background removal tool and a Flutter camera renamer app. A project website was also built to record the field visits and is available at https://cep-project-python.onrender.com.

The project website is a Flask application with a MySQL database, Cloudinary photo storage and five pages: Home, Dashboard, Field Work, Demos and Report. This report also explains the design of the website in detail, including its routing and its photo upload workflow.

The project found that direct, hands-on support helps owners overcome initial hesitation, that follow-up visits are needed to sustain new habits, and that tools are adopted most readily when they fit the existing way of working.

Keywords: Rural Startups, Community Engagement, Digital Adoption, Technical Support, Micro-Enterprises, Photo Processing

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Principal, **Prof. Shashikant Mhalunkar**, for providing the support and academic environment needed to complete our Community Engagement Project.

We are deeply thankful to our Head of Department, **Mr. Pramod L. Shewale**, for his guidance and for providing the facilities and resources required throughout the project.

We would like to express our special thanks to our mentor, **Mrs. Hera Kausar**, for her continuous guidance, valuable suggestions and encouragement during the planning, development and completion of the project.

We are also grateful to the Department of Information Technology and the college administration for giving us the opportunity to undertake this project.

We sincerely thank the entrepreneurs and business owners who welcomed us and cooperated during our field visits. Their participation helped us understand their needs and complete this work.

With Sincere Regards,

Mr. Mergu Pravin Madhukar

Mr. Maurya Chandar Ravindrakumar

Mr. Malik Gulma Gaus

Mr. Machha Yogesh Kamlakar

GUIDANCE BY THE MENTOR

Our mentor, Mrs. Hera Kausar, guided us throughout the Community Engagement Project titled “Technical Support for Rural Startups.” She explained the objectives of the project and helped us plan the field visits to the rural enterprises.

She guided us on how to interact with the business owners in a simple and respectful manner, how to observe their daily work, and how to identify the main difficulty each enterprise was facing.

She also guided us in developing the software tools and the project website so that they would be practical and easy for the owners to use.

Finally, she guided us in documenting the field visits, organising the photographs, analysing our observations and preparing the final report. Her continuous support helped us complete the project in a systematic and meaningful manner.

CERTIFICATE BY THE MENTOR

This is to certify that Mr. Mergu Pravin Madhukar, Mr. Maurya Chandar Ravindrakumar, Mr. Malik Gulma Gaus and Mr. Machha Yogesh Kamlakar, students of T.Y.B.Sc. Information Technology, Semester V, have successfully completed the Community Engagement Project titled “Technical Support for Rural Startups” during the academic year 2026–2027 under my guidance.

The project included field visits to rural enterprises near Bhiwandi, the development of software tools to address the problems observed, and the preparation of a project website.

I appreciate the students for their teamwork, sincere efforts and active participation in completing the project.

Mentor: Mrs. Hera Kausar

Signature: _____

INDEX

Sr.No	Section	Description	Page No.
1		**INTRODUCTION**	**1**
	1.1	Purpose of the Visit	1
	1.2	Background Information	1
	1.3	Scope of the Report	2
	1.4	Problem Statement	2
	1.5	Objectives of the Project Website and Software Tools	3
	1.6	Significance of the Project	3
	1.7	Organisation of the Report	4
2		**LITERATURE REVIEW**	**5**
	2.1	Introduction	5
	2.2	Previous Studies	5
	2.3	Research Gap	5
	2.4	Existing Approaches to Photo Processing and Their Limitations	5
	2.5	Review of Technologies Used	6
3		**METHODOLOGY**	**7**
	3.1	Approach and Tools Used	7
	3.2	Rationale Behind the Methods	7
	3.3	Data Collection and Analysis	7
	3.4	Development Methodology	7
	3.5	Hardware and Software Requirements	8
	3.6	Ethical Considerations	8
4		**FIELD WORK, OBSERVATIONS AND ANALYSIS**	**10**
	4.1	Overview of Field Work	10
	4.2	Detailed Field Visit Records	10
	4.3	Cross-Cutting Observations	14
	4.4	Product Demos: Software Tools Developed	15
	4.5	Project Website	15
	4.6	Project Website Screenshots	16

	4.7	Project Evidence: Field Visit Photographs	19
	4.8	Before and After Summary	20
5		**WEBSITE DESIGN, ROUTING AND IMPLEMENTATION**	**21**
	5.1	Introduction to the Project Website	21
	5.2	System Architecture	21
	5.3	Technology Stack and Dependencies	22
	5.4	Project Structure	22
	5.5	Application Configuration	23
	5.6	Database Design	24
	5.7	Helper Functions	25
	5.8	Routing in the Website	25
	5.9	Detailed Explanation of Each Route	26
	5.10	Photo Upload Workflow	28
	5.11	The Report Page	29
	5.12	Product Demos Catalogue	30
	5.13	Health Check and Keep-Alive Mechanism	30
	5.14	Application Bootstrap and Database Initialisation	31
	5.15	Validation, Error Handling and User Feedback	31
	5.16	Security and Reliability Considerations	32
	5.17	Deployment on Render	32
	5.18	Environment Variables Used by the Application	33
	5.19	Functional Verification	33
	5.20	Limitations and Planned Improvements	34
6		**CONCLUSION AND RECOMMENDATIONS**	**35**
	6.1	Conclusion	35
	6.2	Key Findings	35
	6.3	Significance of Findings	36
	6.4	Recommendations	36
	6.5	Challenges Faced	36
	6.6	Future Scope	37
	6.7	Final Remark	37

7		**REFERENCES**	**38**

CHAPTER 1: INTRODUCTION

1.1 Purpose of the Visit

The main purpose of this project was to give direct, hands-on technical support to micro-businesses and startups in villages near Bhiwandi, Maharashtra. The goal was to help rural entrepreneurs adopt digital tools for payments, inventory tracking, online presence and everyday business operations. Urban businesses take these tools for granted, but village enterprises often cannot use them for lack of hands-on guidance.

Rural micro-enterprises in India work under conditions that make digital adoption difficult. They cannot rely on nearby training centres, stable high-speed internet or a peer network already fluent in digital tools. The gap is not one of awareness but of access to patient, personalised guidance.

The specific objectives of the project were:

- To assess digital readiness: evaluate each enterprise's record-keeping habits, payment methods, familiarity with digital tools and comfort with smartphones.
- To provide practical training: set up tools such as UPI, WhatsApp Business catalogues, Google Business Profiles and Excel-based inventory trackers together with the owner, on their own device.
- To build purpose-built tools: develop simple software utilities for the specific problems observed, where no existing tool fit the workflow.
- To evaluate real adoption: return after one week to check which habits carried over once the students left.
- To document findings: record founder statements, before and after status, visit photographs and details of every enterprise visited.
- To build a project website: present the field records, software demos and the final project report in one place.

1.2 Background Information

Digital adoption among rural micro-enterprises is a well-known challenge in India's development discussions. Government schemes such as Digital India and the UPI payment network have lowered the technical barriers to entry, but lowering a barrier is not the same as crossing it. A shopkeeper still needs someone to sit beside them, on their own phone, and walk through the first few transactions before the tool becomes a habit.

Good digital practices matter for the following reasons:

- Payments: UPI removes the need for exact change, reduces cash-handling risk and allows sales to customers who do not carry cash.
- Discoverability: a Google Business Profile with an accurate location lets customers outside the village find the shop on Search and Maps.
- Inventory: a simple spreadsheet prevents stock-outs and over-ordering, both of which cost money and reputation.
- Photo quality: clean product photos and consistently cropped ID-card photographs are essential for businesses whose output is visual.

1.3 Scope of the Report

This report is based on information collected through field visits, hands-on training sessions, founder interviews and the development of software prototypes. It covers the project objectives, background, detailed field visit records, software demos, the project website, analysis and recommendations.

1.4 Problem Statement

Small enterprises around Bhiwandi, especially those that photograph students for school identity cards, spend most of their working time on repetitive, manual tasks. A photographer who has shot several hundred students in a single day must still rename every file, remove or replace every background and crop every face to the same frame before a single card can be printed. None of these steps requires skill, yet each one consumes hours, and a mistake in any of them leads to a rejected card and a repeated job.

At the same time, the people who run these businesses have very limited access to technical help. Commercial software is usually designed for larger studios and assumes a trained operator and a stable internet connection. Local freelancers are rarely available for a task that must be repeated every week. The problem addressed by this project can therefore be stated in one sentence: **how can students of an Information Technology programme provide practical, low-cost technical support that removes repetitive manual work from rural micro-enterprises and leaves behind a record of what was done?**

1.5 Objectives of the Project Website and Software Tools

Apart from the field visits themselves, the project had a software component. Its objectives were defined before development began so that every feature could be traced back to an observed need:

- To store every field visit as a structured record (enterprise, founder, village, address, date, time, technical support given, founder statement and before and after status) in a relational database instead of in loose notes.
- To present the records in a dashboard that shows the number of visits and the number of distinct villages covered, so that progress is visible at a glance.
- To allow a visit, together with a photograph taken with the founder, to be entered from a simple web form on a phone or a laptop immediately after the visit.
- To keep the uploaded photographs permanently, even though the hosting platform erases its local disk on every restart, by storing them in an external image service.
- To showcase the three software tools as short video demonstrations that any visitor can watch without installing anything.
- To publish the final project report on the same website so that it can be read in the browser and downloaded as a PDF.
- To keep the website available on a free hosting plan, using a health-check route and a keep-alive mechanism so that the first visitor does not wait for a sleeping server.

1.6 Significance of the Project

The project is significant at three levels. For the entrepreneurs, it removed hours of repetitive work and showed that automation can be introduced without replacing their equipment or their way of working. For the students, it provided a real setting in which the theory of programming, databases, web development and image processing had to be applied under practical constraints such as poor connectivity, limited budgets and users who had never used a web application. For the college, it produced a small but complete example of a community engagement programme in which technical education leads directly to local benefit.

The project is also significant because its outputs are reusable. The cropping tool, the background removal utility and the camera renamer can be applied to any school photography business, and the website can be adapted by a future batch of students to record a different set

of visits. This is why the report documents not only what was observed in the field but also how the website was designed, how its routes work and how its source code is organised.

1.7 Organisation of the Report

The report is divided into seven chapters. **Chapter 1** introduces the purpose, background, problem statement, objectives and scope of the project. **Chapter 2** reviews the literature on rural digital adoption and the technologies used. **Chapter 3** describes the methodology, the development approach, the system requirements and the ethical considerations. **Chapter 4** records the four field visits, the cross-cutting observations, the three software tools, the project website and the evidence collected. **Chapter 5** explains the design and implementation of the website in detail, covering its architecture, configuration, database, routing, photo upload workflow, report page, health check and deployment. **Chapter 6** gives the conclusion, findings, challenges, future scope and recommendations. **Chapter 7** lists the references.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The literature on rural digital adoption in India has grown considerably over the past decade. Most studies agree on one theme: awareness of digital tools is not the same as adoption, and adoption is not the same as sustained use. The gap between these three states is where field-level interventions matter most.

2.2 Previous Studies

Reports from the Telecom Regulatory Authority of India and NASSCOM have repeatedly shown that rural internet penetration has risen sharply, but business use of digital tools lags behind personal use [5][6]. Studies of UPI adoption in tier-3 towns consistently find that the first successful transaction is the biggest predictor of continued use, which supports the hands-on-first approach adopted in this project [2].

Research on micro-enterprise digitisation also stresses fit over sophistication. Tools that match an owner's existing workflow are adopted far more readily than tools that require a complete change of habit.

2.3 Research Gap

What is less documented is the day-to-day reality of a technical support visit: the specific problems that emerge only when one sits with an entrepreneur and watches them work. This report aims to fill part of that gap by documenting four such visits in detail, along with the purpose-built tools that resulted.

2.4 Existing Approaches to Photo Processing and Their Limitations

Three approaches to preparing identity-card photographs were seen during the field work. The first is manual editing in a general-purpose image editor: each photograph is opened, cropped, saved and closed. This gives full control, but on a shoot of 500 students it occupies a full working day, as the visit to Digital ID Solutions showed. The second approach is the use of online background removal websites. These are convenient for a single image, but they require an upload, a wait and a download for every photograph, they depend on an internet connection, and free plans often limit the number or the quality of images. The third approach is the use of commercial desktop software with batch features. This is effective, but licence cost and the learning curve can put it out of reach of a small village business.

The limitation common to all three approaches is that they treat the photograph as an individual task. An ID-card business, by contrast, receives photographs in batches of hundreds, all of which need the same treatment. A tool designed around the batch, which runs locally, keeps the original file names and needs no training, fits the actual workflow far better. This observation shaped the design of the three tools described in Section 4.4.

2.5 Review of Technologies Used

The technologies used in the project were selected for being free, well documented, widely used and able to run on modest hardware. They are reviewed briefly here and their use in the project is described in Chapter 5.

- **Flask** is a lightweight Python web framework that maps URLs to Python functions and renders HTML through the Jinja2 template engine [8]. It was chosen because a complete application can be written in a single readable file, which suits a small team and makes the code easy to explain.
- **SQLAlchemy and Flask-SQLAlchemy** provide an object-relational mapper, so that a table in the database can be handled as a Python class and queries can be written in Python instead of raw SQL [9][10].
- **MySQL with PyMySQL** was used as the database. MySQL is a mature relational database, and PyMySQL is a pure-Python driver that lets SQLAlchemy connect to it without compiled extensions [11]. The database was hosted on a managed MySQL service, which removes the need to administer a server [17].
- **Cloudinary** is a cloud service for storing and delivering images. Its Python SDK uploads a file with one function call and returns a permanent HTTPS address [12].
- **Render and Gunicorn** were used for deployment. Render builds and runs the application from the project repository, and Gunicorn is the production-grade server that runs the Flask application behind it [13][14].
- **RetinaFace** is a single-stage face detector that locates faces and facial landmarks accurately even in crowded or poorly lit images [15]. It is the basis of the auto-crop tool, while **Pillow** performs the actual cropping and resizing [16].
- **Tailwind CSS** is a utility-first styling framework that was used to make the pages responsive, so that the website works on both a phone and a desktop screen [18].

CHAPTER 3: METHODOLOGY

3.1 Approach and Tools Used

The project used a mixed-method approach combining qualitative observation with documentation of results. Each enterprise was visited in person, and the interaction followed a fixed sequence: initial conversation, observation of the daily workflow, identification of the main problem, and collaborative setup or development of the solution. The methods used were:

- Direct observation of the founder's daily operations, record-keeping and payment methods.
- Semi-structured interviews to understand the founder's stated difficulties.
- Hands-on setup of digital tools on the founder's own device.
- Development of purpose-built software where no existing tool fit the workflow.
- Photographic documentation of each visit, with the founder's consent.
- A structured database and website to record and present the outcomes.

3.2 Rationale Behind the Methods

Each method was chosen for a specific reason. Observation captured details that founders would not think to mention, such as the app they hesitated on, the folder of paper receipts, or the manual step repeated a hundred times a day. Interviews captured the founder's own view of the problem. Hands-on setup ensured the founder saw the tool working on their own data. Purpose-built development covered cases where existing tools did not fit the workflow.

3.3 Data Collection and Analysis

Data was collected through handwritten notes, photographs and a structured form filled in during or immediately after each visit. The analysis was thematic. It looked across the four visits for recurring patterns in the type of problem, the founder's response to the intervention, and the extent to which the solution was adopted.

3.4 Development Methodology

The software was developed iteratively, in short cycles, because the requirements were discovered only during the visits. Each cycle had five steps. First, the problem was observed and described in the founder's own words. Second, a minimal prototype was written that solved only that problem. Third, the prototype was tried on real photographs, usually on the founder's

own files. Fourth, the founder's comments were noted and the prototype was corrected. Fifth, the working tool was handed over and checked again during the follow-up.

The website was built in the same way, one feature at a time, and grew into the present five-page application: Home, Dashboard, Field Work, Demos and Report. Each feature was run locally, tested by submitting real records, and only then deployed. This approach avoided building features that were never needed and kept the final code small enough to be explained in this report.

3.5 Hardware and Software Requirements

The tools were intentionally designed to run on the kind of equipment that the entrepreneurs already own. The requirements are summarised in the table below.

Table 3.1: Hardware and software requirements

Item	Requirement
Development computer	Laptop or desktop computer able to run the face detection model
Operating system	Any desktop system that runs Python for the photo tools; Android (and iOS) devices for the Flutter camera renamer app
Programming languages	Python 3 for the website and photo tools; Dart (Flutter) for the mobile app
Web framework and libraries	Flask, Flask-SQLAlchemy, PyMySQL, Cloudinary SDK, Pillow, RetinaFace, withoutbg
Database	MySQL (managed instance with TLS enabled)
Browser	Any current browser, such as Chrome, Edge or Firefox, on a phone or a computer
Internet	Needed only for the website; the photo tools run locally without an internet connection
Hosting	Render web service running the application through Gunicorn

3.6 Ethical Considerations

Because the project involved real businesses and photographs of real people, the following rules were followed throughout. The founder's consent was obtained before any visit photograph was taken, and the statements recorded in this report are those of the founders, presented as they were given. The entrepreneurs were free to decline or stop using any tool. Because the photo tools run locally, the photographs of students that they process do not need

to be uploaded to any online service, which is important when the subjects are school children. The project website stores only the field record and the visit photograph taken with the founder's consent.

CHAPTER 4: FIELD WORK, OBSERVATIONS AND ANALYSIS

4.1 Overview of Field Work

The field work was carried out across four enterprises in Bhiwandi and the surrounding villages of Shelar, Kariwali and Khoni Gaon during September and October 2026. The team consisted of four T.Y. B.Sc. Information Technology students working under the guidance of the project mentor.

Each visit followed a three-stage structure. In the first stage, the team sat with the founder and observed the daily operations. In the second stage, the team and the founder identified the most significant problem in the workflow. In the third stage, the solution, whether an existing tool or a purpose-built one, was demonstrated and set up on the founder's own device before the team left.

A recurring observation across all four visits was that the founders were not lacking in awareness of digital tools. They already knew the kind of solution they needed. What they lacked was the technical know-how to set it up, or a tool that fit their exact workflow.

4.2 Detailed Field Visit Records

4.2.1 Srividyaniketan Classes, Bhiwandi

Startup Name	Srividyaniketan Classes
Founder	Niraj
Village	Bhiwandi
Date of Visit	03 October 2026
Time of Visit	11:20 AM

Background of the Enterprise

Srividyaniketan Classes is a coaching institute in Bhiwandi. It conducts regular classes in multiple subjects and manages a small teaching staff along with its students. Like many institutes of its size, its administrative work has been handled manually, with separate registers for attendance, fee collection and internal records.

Observed Problem

The administrative burden fell almost entirely on the founder. Attendance for every student and teacher was tracked on paper registers, and there was no central record of the instalments

paid by students. When a parent asked about a child's fee status, the founder had to cross-check several registers by hand.

Founder's Statement

"Has to manually track the attendance of every student and teacher, and there is no track of installments paid by the students. The founder wants to upload the marks online so they are accessible anywhere."

Solution Designed

A class management web application was built to bring the institute's administrative work into one place. It tracks attendance, fee instalments, student marks and basic student records in a single dashboard that can be opened from any smartphone or browser.

Outcome

The web application was demonstrated and handed over during the visit. In the same session the founder was able to log in, register a student, mark attendance and record a fee instalment.

4.2.2 Baba ID Cards Solutions, Shelar

Startup Name	Baba ID Cards Solutions
Founder	Shridhar Siripuram
Village	Shelar
Date of Visit	29 September 2026
Time of Visit	01:14 PM

Background of the Enterprise

Baba ID Cards Solutions runs a school-photography business. The founder travels to schools across the Bhiwandi region to photograph students for ID cards and school records. Each school visit produces hundreds of photographs, which must be transferred, sorted and named before delivery.

Observed Problem

The founder's workflow relied on a DSLR camera and a laptop. After each school visit, the photographs were transferred to the computer and renamed manually, one by one, to keep a consistent sequence.

Founder's Statement

"The founder needs to click the photos from every school. For that purpose he has to take a bag and a large camera. The need is that the camera has a feature that names photos with a forwarding number as the image name."

Solution Designed

A mobile companion app was developed using Flutter that captures a photo, saves it directly into a chosen folder, and renames it with a DSLR-style serial number (IMG_0001, IMG_0002 and so on). The number continues from the last file in the folder, so the sequence stays correct across sessions.

Outcome

The app was installed on the founder's phone during the visit. Ten photographs were captured in sequence and checked for correct filenames in the correct folder.

4.2.3 V Tech ID Solutions, Kariwali

Startup Name	V Tech ID Solutions
Founder	Venkatesh Chitiken
Village	Kariwali
Date of Visit	24 September 2026
Time of Visit	02:59 PM

Background of the Enterprise

V Tech ID Solutions is a small ID-card service business in Kariwali. The founder processes photographs submitted by schools, colleges and offices for ID cards and other official documents. A typical week involves processing between 200 and 500 photographs.

Observed Problem

The main task was background replacement: taking each submitted photo and replacing its background with a clean, standard colour before printing. The founder did this manually by uploading, waiting, selecting and downloading, and then repeating the steps for every image.

Founder's Statement

"Facing problem in manually selecting every photo — upload, change background, download, then repeat. This has to be done for each photo individually."

Solution Designed
A bulk background removal tool was built using the withoutbg model. It takes a folder of input images, removes the background from each in a single run, and saves the cleaned results in an output folder with the original filenames intact.

Outcome
The tool was tested on a batch of 30 photographs during the visit. The batch finished in under two minutes, compared with the founder's estimate of about ninety minutes by hand.

4.2.4 Digital ID Solutions, Khoni Gaon

Startup Name	Digital ID Solutions
Founder	Santosh Chitiken
Village	Khoni Gaon
Date of Visit	24 September 2026
Time of Visit	05:04 PM

Background of the Enterprise
Digital ID Solutions is based in Khoni Gaon and provides ID-card and school photography services. The founder handles the whole process, from photographing students to producing finished ID cards. Each school visit produces a large set of photographs that must be processed into a consistent ID-card format.

Observed Problem
The bottleneck was cropping. After every school shoot, the founder opened each photograph in an editor, adjusted the crop to frame the face and shoulders correctly, and saved it. On a recent shoot the founder had photographed 500 students at a single school, and each photo needed manual cropping.

Founder's Statement
"The founder clicked 500 photos of students at a school and then had to crop the entire batch manually by adjusting the zoom and shoulder position for each individual photo."

Solution Designed
An auto-crop web application was built using RetinaFace for face detection and Pillow for image processing. It accepts a batch of photographs, detects each face, estimates the shoulder line, and crops every photo to a standard ID-card ratio with the face and shoulders correctly placed.

Outcome

The tool was demonstrated on a set of test photographs during the visit. A batch of 50 photos was processed in about thirty seconds with consistently framed results. For a 500-photo shoot, it reduces the processing time from a full day to a few minutes.

Summary of Field Visits

Startup	Founder	Village	Date	Time
Srividyaniketan Classes	Niraj	Bhiwandi	03-Oct-2026	11:20 AM
Baba ID Cards Solutions	Shridhar Siripuram	Shelar	29-Sep-2026	01:14 PM
V Tech ID Solutions	Venkatesh Chitiken	Kariwali	24-Sep-2026	02:59 PM
Digital ID Solutions	Santosh Chitiken	Khoni Gaon	24-Sep-2026	05:04 PM

4.3 Cross-Cutting Observations

Looking across the four visits, several patterns emerge that are not visible in any single record but become clear when the four are considered together.

- The problem is known; the tool is missing: in every visit the founder already understood the problem. They did not need to be convinced of the value of digitisation. They lacked a tool that fit their workflow, or the know-how to set up a generic one.
- Repetitive manual work is the main bottleneck: three of the four enterprises had the same underlying problem, the repetitive manual processing of many photographs. When shown the automated version, each founder immediately saw its value.
- The photo-services cluster is an emerging local market: three of the four enterprises work in ID-card and school photography. The Bhiwandi region has many schools and colleges, and demand for ID-card services is high.
- Fit beats sophistication: in every case, the solution that was adopted was the one that needed the least change to the founder's existing workflow.
- Follow-up is not optional: even the most promising demonstrations were followed by questions about edge cases and long-term data safety. Without a follow-up, some of these questions would have gone unanswered.

4.4 Product Demos: Software Tools Developed

During the field work, three recurring problems emerged that could not be solved by training alone. In response, three small software tools were developed. Each tool is demonstrated with a video walkthrough on the project website at https://cep-project-python.onrender.com/demos.

Demo 1: Auto Face and Shoulder Cropping Web App

Developed in response to Record 4 (Digital ID Solutions). The tool accepts a batch of photographs, detects every face using RetinaFace, estimates the shoulder line, and crops each photo to a standard ID or portrait ratio. It handles single and batch uploads and saves the output with the original filenames so the founder's folder structure is preserved. Technology used: Python, RetinaFace, Pillow, Flask.

Demo 2: Bulk Background Removal Utility

Developed in response to Record 3 (V Tech ID Solutions). The tool processes an entire folder of photographs in one run, removes the background from each image, and saves the result with a clean white or transparent background. It uses the withoutbg model, which runs locally without an internet connection. Technology used: Python, withoutbg, Pillow.

Demo 3: Smart Camera Renamer (Flutter App)

Developed in response to Record 2 (Baba ID Cards Solutions). The app captures a photo, saves it directly into a chosen folder, and renames it with a DSLR-style serial number (IMG_0001, IMG_0002 and so on). The sequence continues across sessions, so the founder can return to a school a week later and carry on where the last batch ended. Technology used: Flutter, Dart, EXIF metadata.

4.5 Project Website

To make the project's findings, training materials and software tools available beyond the immediate team, a web application was built using Flask, MySQL and Jinja2 templates styled with Tailwind CSS, and deployed on a cloud hosting service. The website is live at https://cep-project-python.onrender.com. The main components are listed below.

Component	Technology	Purpose
Backend framework	Flask 3.0	Routing, requests and page templates
Database	MySQL	Storage of field visit records
Frontend	Jinja2 and Tailwind CSS	Responsive web pages
Photo storage	Cloudinary	Storage of uploaded visit photos
Deployment	Render and Gunicorn	Production hosting

The website has an overview page with the project objectives, team and methodology; a dashboard with key figures and a table of all field visits; a field work page that shows each visit as a card with photographs, date, before and after status and the founder's statement; a demos page with video walkthroughs of the three software tools; and a report page where the complete project report can be read in the browser and downloaded as a PDF.

4.6 Project Website Screenshots

The screenshots below show the main pages of the deployed project website.

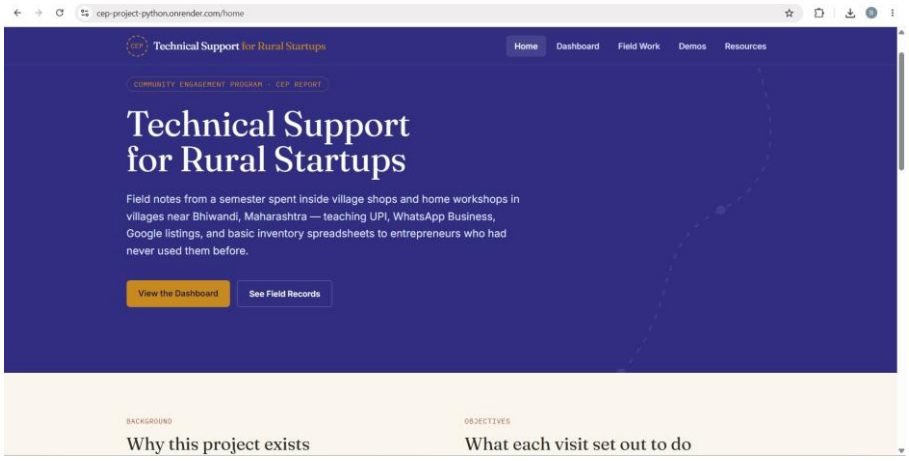


Figure 4.1: Project website: Home page.

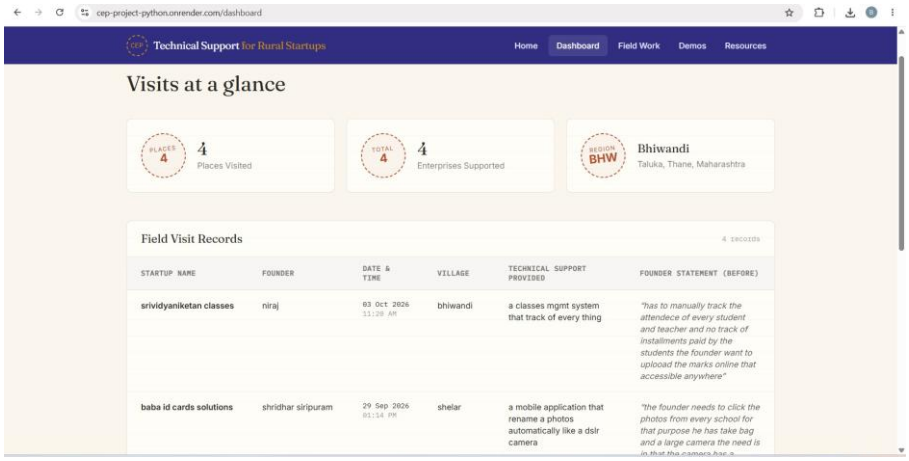


Figure 4.2: Project website: Dashboard of field visit records.

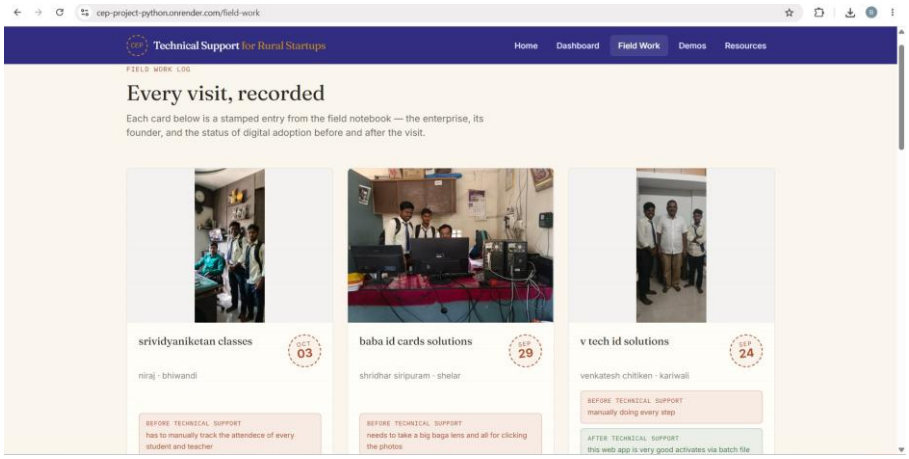


Figure 4.3: Project website: Field Work log with visit cards.

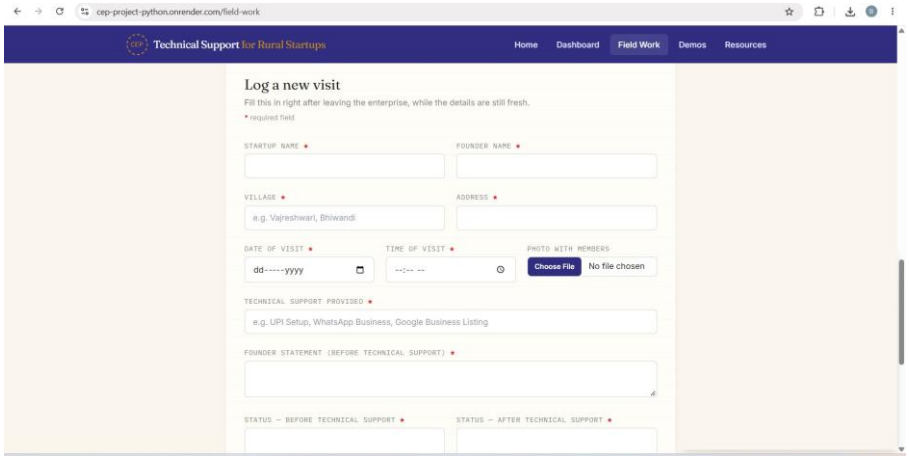


Figure 4.4: Project website: Form to log a new visit.

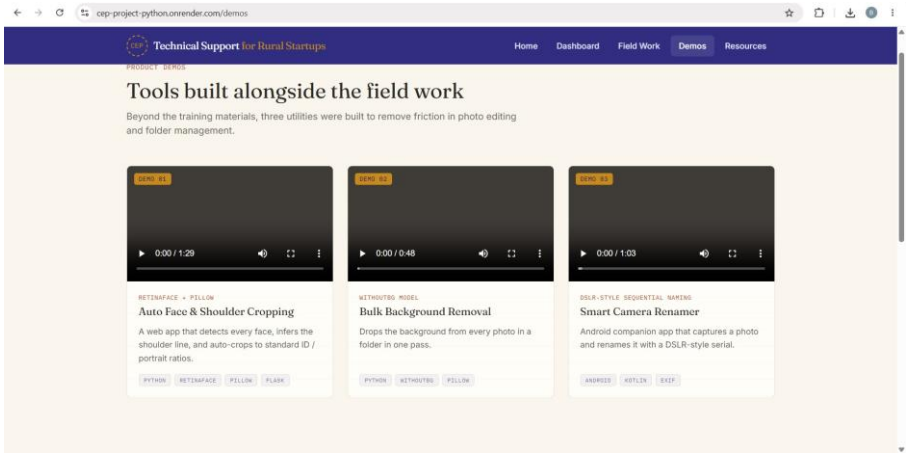


Figure 4.5: Project website: Product Demos page.

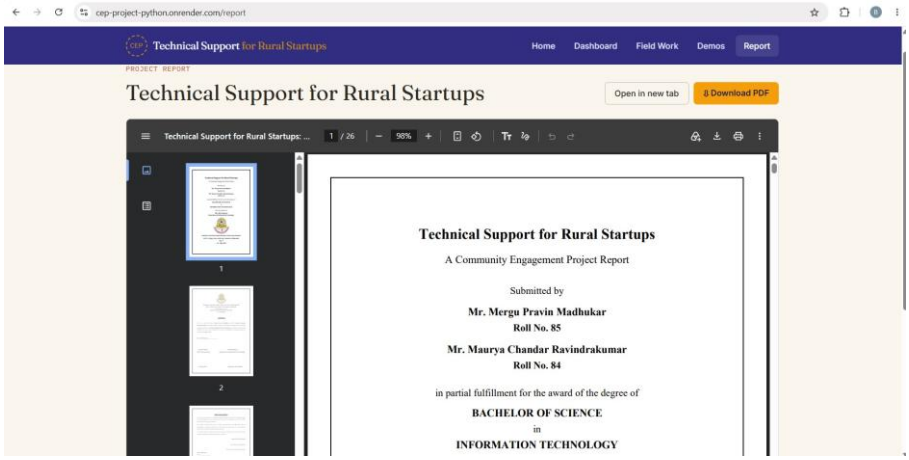


Figure 4.6: Project website: Report page.

4.7 Project Evidence: Field Visit Photographs

The photographs below were taken during the field visits, with the consent of the owners.



Figure 4.7: Srividyaniketan Classes, Bhiwandi.

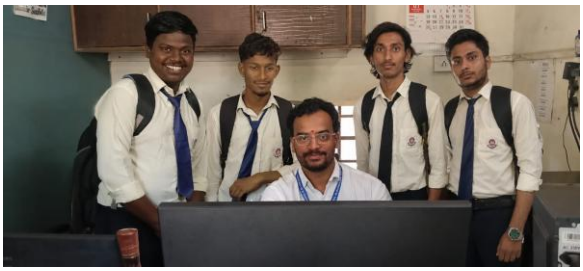


Figure 4.8: Baba ID Cards Solutions, Shelar.



Figure 4.9: V Tech ID Solutions, Kariwali.



Figure 4.10: Digital ID Solutions, Khoni Gaon.

4.8 Before and After Summary

The table below brings together the situation found at each enterprise and the situation after the intervention, using the observations recorded in Section 4.2. It shows that in each case the change was measured against the way the founder already worked.

Table 4.1: Before and after comparison of the four enterprises

Enterprise	Before	After
Srividyaniketan Classes	Attendance, fees and records kept in separate paper registers; fee status checked by hand across registers	Single web dashboard for attendance, fee instalments, marks and student records, opened from any smartphone
Baba ID Cards Solutions	Photographs renamed one by one on the laptop after each school visit	Flutter app captures and renames photographs in a continuing IMG_0001 sequence
V Tech ID Solutions	Backgrounds changed photo by photo (upload, edit, download); about ninety minutes for 30 photos by the founder's estimate	Folder-level bulk removal; 30 photos processed in under two minutes
Digital ID Solutions	Each of 500 photographs cropped manually; roughly a full day of work	Batch auto-crop; 50 photos processed in about thirty seconds

CHAPTER 5: WEBSITE DESIGN, ROUTING AND IMPLEMENTATION

5.1 Introduction to the Project Website

The project website is the digital face of the Community Engagement Project. It is live at https://cep-project-python.onrender.com and has five pages that can be reached from the navigation bar at the top of every screen: **Home**, **Dashboard**, **Field Work**, **Demos** and **Report**. The Home page introduces the project, its background and its objectives. The Dashboard summarises the field visits in numbers and in a table. The Field Work page lists every visit as a card and also contains the form used to add a new visit. The Demos page plays the video walkthroughs of the three software tools. The Report page shows this project report inside the browser and lets the visitor open it in a new tab or download it as a PDF.

The whole server side of the website is written in one Python file, `app.py`, together with a small `models.py` file that defines the database table. This chapter explains that file section by section so that a reader who has never seen the code can understand what each part does and why it was written that way.

5.2 System Architecture

The website follows the standard three-part structure of a web application. The **browser** sends a request, the **Flask application** decides what to do with it, and the application then reads from or writes to the **data services** (the MySQL database, Cloudinary and the static folder) before returning an HTML page. Figure 5.1 shows how these parts are connected.

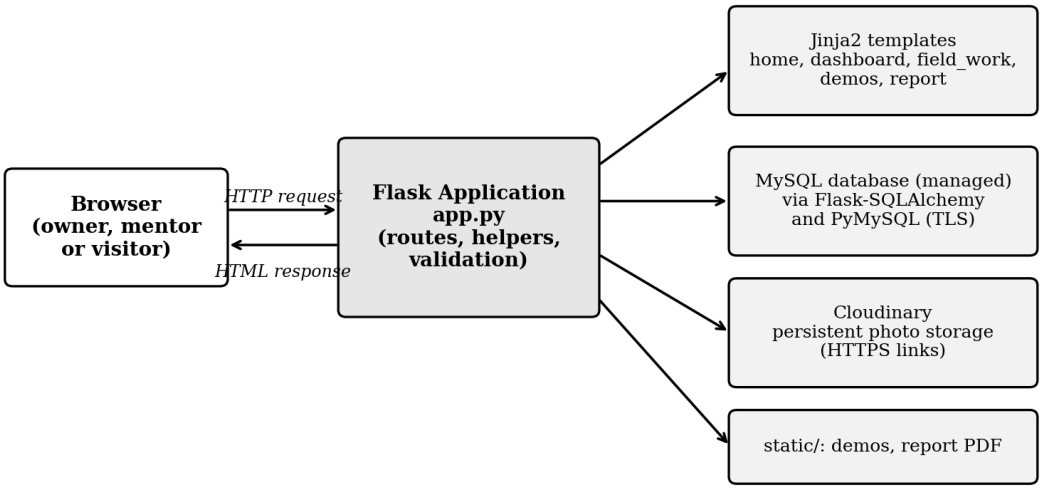


Figure 5.1: Architecture of the project website.

Three design decisions follow from this architecture. First, all business logic stays in one place, the Flask application, so there is only one file to read and debug. Second, the database stores text only, and photographs are kept in Cloudinary with only their web address saved in the database, which keeps the database small and the photographs safe from server restarts. Third, the pages are rendered on the server from Jinja2 templates, so the browser receives ready-made HTML and the site works even on slow phones.

5.3 Technology Stack and Dependencies

The table below lists every Python module that `app.py` imports and the role it plays in the application.

Table 5.1: Modules used by app.py and their roles

Module	Role in the application
`os`	Reads environment variables and builds file paths for the static and upload folders
`threading`, `time`, `urllib.request`	Run the background keep-alive thread that pings the website at fixed intervals
`datetime`	Converts the date and time typed in the form into Python date and time objects
`flask`	Provides the `Flask` class, `render_template`, `request`, `redirect`, `url_for`, `flash`, `send_from_directory` and `abort`
`werkzeug.utils.secure_filename`	Cleans a file name so that it is safe to save on disk
`sqlalchemy` (`func`, `text`)	Builds the distinct-village count and the `SELECT 1` health query
`flask_sqlalchemy` (through `models.py`)	Supplies the `db` object and the `Startup` table class
`cloudinary`, `cloudinary.uploader`	Uploads photographs to the cloud and returns a permanent link
PyMySQL (driver)	Lets SQLAlchemy talk to the MySQL server using a `mysql+pymysql://` address

5.4 Project Structure

The files and folders that `app.py` refers to give the following structure for the project:

- `app.py`: the Flask application containing the configuration, helper functions and all routes.
- `models.py`: defines the `db` object and the `Startup` model that represents one field visit.
- `templates/`: the Jinja2 pages `home.html`, `dashboard.html`, `field_work.html`, `demos.html` and `report.html`.
- `static/uploads/`: the local folder used for photographs only when Cloudinary is not configured, for example during development.
- `static/demos/`: the demo videos, their poster images and the screenshot of the class management website.
- `static/CEP_Report_Technical_Support_Rural_Startups.pdf`: the final project report that is shown on the Report page.

5.5 Application Configuration

The first part of `app.py` creates the Flask object and sets its configuration. Every setting is described below.

5.5.1 Base paths and allowed file types

`BASE_DIR` stores the absolute path of the folder that contains `app.py`. All other paths are built from it, so the application finds its folders correctly no matter from which directory it is started. `UPLOAD_FOLDER` points to `static/uploads`, and `ALLOWED_EXTENSIONS` is the set `png`, `jpg`, `jpeg`, `gif` and `webp`. Only files whose extension is in this set are accepted as visit photographs.

5.5.2 Secret key

Flask needs a secret key to sign the session cookie that carries flash messages from one request to the next. The key is read from the `SECRET_KEY` environment variable, with a development value as a fallback. On the live website the key should be set to a long random value in the hosting dashboard.

5.5.3 Database connection

The database address starts with `mysql+pymysql://`, which tells SQLAlchemy to use the PyMySQL driver, and is followed by the user, password, host, port and database name. These five parts (`DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` and `DB_NAME`) are read from environment variables so that the same code can be pointed at a different database without editing it. The managed MySQL service requires an encrypted connection, so the application also sets `ssl` in `connect_args`. PyMySQL does not accept an `ssl_mode` argument in the URL,

which is a convention of another driver, and it takes a plain `ssl` dictionary instead. The setting `DB_SSL_MODE` can be set to `DISABLED` to switch encryption off for a local database.

5.5.4 Engine options

Three options are passed to the SQLAlchemy engine to keep the database connection healthy on a free hosting plan:

- `pool_pre_ping` is set to true, so each pooled connection is tested before use, and a dead connection is replaced instead of causing an error.
- `pool_recycle` is set to 280 seconds, so idle connections are closed and reopened before the server drops them.
- `connect_timeout` is set to 10 seconds, so a slow database cannot hold a request for a long time.

`SQLALCHEMY_TRACK_MODIFICATIONS` is set to false because change tracking is not needed and would waste memory.

5.5.5 Upload limit

`MAX_CONTENT_LENGTH` is set to 8 MB. Flask rejects any request larger than this before the application code runs, which protects the server from very large uploads. The call `db.init_app(app)` then connects the database object from `models.py` to this application.

5.5.6 Cloudinary configuration

Render erases its local disk whenever the service restarts or is redeployed, so photographs saved there would eventually disappear. For this reason photographs are stored in Cloudinary. The Cloudinary library configures itself automatically from the `CLOUDINARY_URL` environment variable. The application therefore only checks whether the variable exists and stores the result in `USE_CLOUDINARY`. If it exists, `cloudinary.config(secure=True)` makes sure that all returned links use HTTPS.

5.6 Database Design

The database has one main table, created from the `Startup` class in `models.py`. Each row is one field visit. The columns used by the application are listed below.

Table 5.2: Fields of the Startup table (one row per field visit)

Field	Type	Meaning
`startup_name`	Text	Name of the enterprise visited

Field	Type	Meaning
`founder_name`	Text	Name of the founder or owner
`photo_filename`	Text	Cloudinary link of the visit photograph, or a local file name; empty if no photograph was uploaded
`village`	Text	Village or town where the enterprise is located
`address`	Text	Address of the enterprise
`date_visited`	Date	Date of the visit
`time_visited`	Time	Time of the visit
`technical_support`	Text	Technical support given during the visit
`statement_before`	Text	Founder's statement of the problem before the support
`before_tech_support`	Text	Status of the enterprise before the support
`after_tech_support`	Text	Status of the enterprise after the support

The table is created automatically when the application starts, as explained in Section 5.14, so no separate database script has to be run.

5.7 Helper Functions

Three small functions keep the routes short and readable.

- `allowed_file(filename)` returns true when the file name contains a dot and the text after the last dot, converted to lower case, is in `ALLOWED_EXTENSIONS`. Converting to lower case means that `PHOTO.JPG` and `photo.jpg` are treated alike.
- `parse_date(value)` converts a text such as `2026-10-03`, which is what an HTML date field sends, into a Python date.
- `parse_time(value)` converts a text such as `11:20` or `11:20:00` into a Python time. It counts the colons to decide whether seconds are present, because different browsers send the time field in either form.

5.8 Routing in the Website

Routing means deciding which Python function answers which web address. In Flask this is done with the `@app.route()` decorator placed above a function. When a request arrives, Flask compares its path with the registered routes, calls the matching function and sends back whatever that function returns. If no route matches, Flask replies with a 404 error. The website registers the routes shown in Table 5.3.

Table 5.3: Routes of the project website

URL	Methods	Function	What it does
`/` and `/home`	GET	`home()`	Renders `home.html`, the introduction to the project
`/dashboard`	GET	`dashboard()`	Lists all visits, newest first, and shows the totals
`/field-work`	GET, POST	`field_work()`	GET shows the visit cards and the form; POST saves a new visit
`/demos`	GET	`demos()`	Renders the product demo videos
`/report`	GET	`report()`	Shows the project report PDF inside the page
`/report/file`	GET	`report_file()`	Serves the PDF itself for display inside the browser
`/report/download`	GET	`download_report()`	Serves the same PDF as a file download
`/healthz`	GET	`healthz()`	Checks the database and returns ok or an error

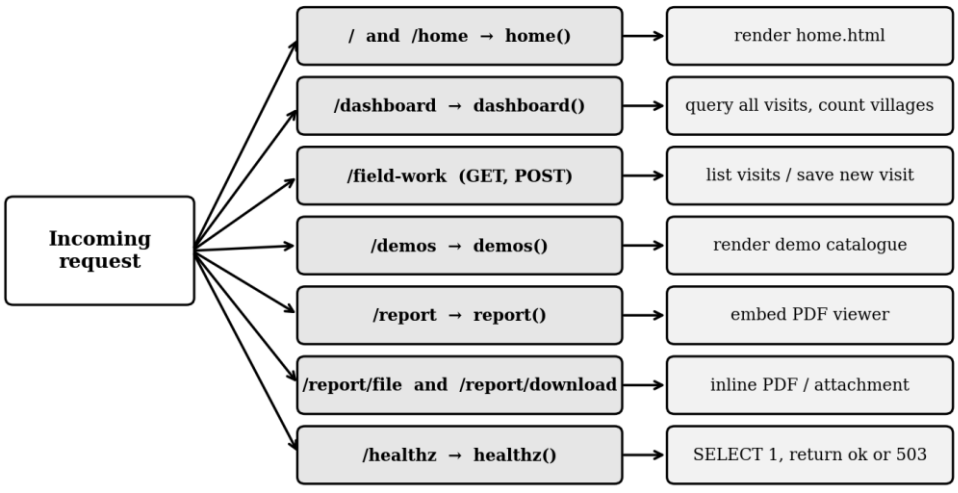


Figure 5.2: Routing map of the website.

5.9 Detailed Explanation of Each Route

5.9.1 Home route: / and /home

The function `home()` has two route decorators, so that both the bare address of the website and `/home` show the same page. It does nothing except call `render_template("home.html")`.

Having two addresses means that a visitor who types only the domain name and a visitor who follows the Home link in the navigation bar both reach the same place.

5.9.2 Dashboard route: /dashboard

The dashboard function performs one database query to fetch every visit, ordered by `date_visited` from newest to oldest, and another to count the number of different villages with `func.count(func.distinct(Startup.village))`. If the table is empty the count is replaced by zero. The function then passes three values to `dashboard.html`: the list of visits, the number of places visited and the total number of visits, which is simply the length of the list. The template turns these into the figures and the table seen on the Dashboard page.

5.9.3 Field Work route: /field-work

This is the most important route because it accepts both **GET** and **POST** requests. When a visitor opens the page, the browser sends a GET request, and the function fetches all visits, newest first, and renders `field_work.html`, which shows them as cards together with the form. When the form is submitted, the browser sends a POST request to the same address, and the function does the following:

- Reads the optional file named `photo_with_members` and uploads it as described in Section 5.10.
- Reads the text fields `startup_name`, `founder_name`, `village`, `address`, `technical_support`, `statement_before`, `before_tech_support` and `after_tech_support`, removing extra spaces from each with `strip()`.
- Converts the date and time fields with `parse_date` and `parse_time`.
- Creates a `Startup` object, adds it to the database session and commits it.
- Shows a green success message with `flash()`, or an error message if a field was missing or invalid.
- Redirects to `/field-work` with `redirect(url_for("field_work"))`.

The final redirect follows the Post/Redirect/Get pattern. Without it, pressing the refresh button after submitting the form would ask the browser to send the same POST again and would create a duplicate record. With the redirect, the browser ends on an ordinary GET page and refreshing it is harmless.

5.9.4 Demos route: /demos

The demos function renders `demos.html` and passes it two things: the list returned by `_demo_catalogue()` and the address of the class management website screenshot. Both can be replaced through environment variables, as described in Section 5.12.

5.9.5 Report routes: /report, /report/file and /report/download

The three report routes work together and are explained in Section 5.11.

5.9.6 Health route: /healthz

The health route is explained in Section 5.13.

5.10 Photo Upload Workflow

Uploading a photograph is the only place where the website handles a file, so it is designed with care. Figure 5.3 shows the decisions taken for every submitted form.

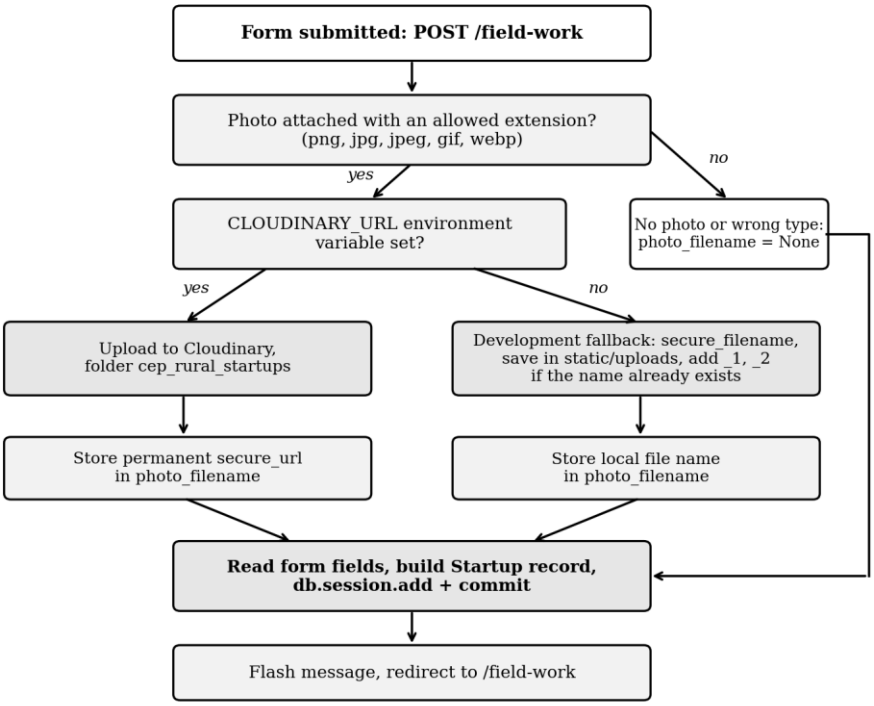


Figure 5.3: Photo upload workflow of the Field Work form.

The steps are as follows.

Step 1: Check the file

The application reads `request.files.get("photo_with_members")`. A photograph is processed only if a file was actually chosen, the file name is not empty and `allowed_file()` accepts its extension. In every other case `photo_filename` stays empty, and the visit is still saved without a photograph. This makes the photograph optional.

Step 2: Upload to Cloudinary when configured

If `USE_CLOUDINARY` is true, the file is sent with `cloudinary.uploader.upload(photo, folder="cep_rural_startups")`. Cloudinary stores the image in that folder and returns a dictionary. The application takes the value `secure_url`, which is a permanent HTTPS address, and saves it in the `photo_filename` column. The database therefore holds a link, and the image itself lives in Cloudinary, where it survives every restart of the website.

Step 3: Local fallback during development

If Cloudinary is not configured, the file is saved in `static/uploads`. The file name is first cleaned with `secure_filename()`, which removes folder separators and unsafe characters. If a file with the same name already exists, a counter is added (`name_1.jpg`, `name_2.jpg` and so on) until an unused name is found, so no earlier photograph is ever overwritten. Only the final file name is stored in the database. This route is meant for development, because files saved on the hosting platform's disk are not permanent.

Step 4: Save the record

After the photograph is handled, the text fields are read and the `Startup` record is committed to the database. The success message names the enterprise, for example "Field record for V Tech ID Solutions saved successfully."

5.11 The Report Page

The **Report** page is the last item in the navigation bar and is where the final project report is published. It lets visitors read the complete report on the website itself, without having to ask the team for a copy. Figure 4.6 in Chapter 4 shows the page: it has a small label "Project Report", the project title, two buttons named **Open in new tab** and **Download PDF**, and below them a PDF viewer that displays the report page by page.

Three routes make this possible.

- **`/report`** renders `report.html`, the page that contains the heading, the two buttons and the embedded viewer.
- **`/report/file`** uses `send_from_directory()` with `as_attachment=False` and the MIME type `application/pdf`. The browser therefore displays the PDF instead of saving it, which is what the embedded viewer and the Open in new tab button need.
- **`/report/download`** serves the same file with `as_attachment=True`, so the browser saves it to the computer. This route is used by the Download PDF button.

All three routes first call the helper `_report_path_ok()`, which checks that the file `CEP_Report_Technical_Support_Rural_Startups.pdf` really exists in the `static` folder. If it does not, the route calls `abort(404)` and the visitor sees a normal "not found" page instead of a broken viewer. The file name is kept in a single constant, `REPORT_FILENAME`, so replacing the report with a new version only requires copying a new PDF with the same name into the `static` folder. No change to the code is needed.

5.12 Product Demos Catalogue

The function `_demo_catalogue()` returns a list of three dictionaries, one for each software tool. Each dictionary holds a `slug`, a `number`, a `title`, a `tagline`, a `description`, a `stack` list of technologies, a `video` address and a `poster` image. The Demos page loops over this list to build one card per tool, so adding a fourth tool means adding one more dictionary and no new HTML.

Table 5.4: Entries of the product demo catalogue

No.	Title	Technology stack
01	Auto Face and Shoulder Cropping	Python, RetinaFace, Pillow, Flask
02	Bulk Background Removal	Python, withoutbg, Pillow
03	Smart Camera Renamer	Flutter, Dart, EXIF

The video address of each entry is read from an environment variable (`DEMO_VIDEO_CROP_URL`, `DEMO_VIDEO_BG_URL` and `DEMO_VIDEO_RENAMER_URL`). If the variable is not set, the application falls back to a file in `static/demos`. Large videos can therefore be hosted on an external service and linked by address, while small ones are served from the website itself. The helper `url_for("static", filename=...)` builds the correct address of each static file.

5.13 Health Check and Keep-Alive Mechanism

The free hosting plan stops a web service after about fifteen minutes without visitors, and the next visitor then waits while the service starts again. Two features reduce this delay.

The route `/healthz` runs the query `SELECT 1` through the database session. If the query succeeds it returns the text "ok" with status 200. If it fails it returns "db unavailable" with status 503. Because the query touches the database, a single request keeps both the web service and the database connection active. The route is cheap, so an external uptime monitor can call it every few minutes.

In addition, the function `_keep_alive(url, interval=600)` runs in a background thread. It sleeps for 600 seconds, requests the health address through the public website address, and repeats. Any error during the request is ignored so that a single failed request can never stop the thread. The thread is started only when an address is available, either from `KEEP_ALIVE_URL` or from `RENDER_EXTERNAL_URL`, which Render sets automatically. On a local computer neither variable exists, so no thread is started. Setting `KEEP_ALIVE` to `0` switches the feature off. The thread is created as a daemon, so it never prevents the application from shutting down.

5.14 Application Bootstrap and Database Initialisation

The function `init_db()` opens an application context and calls `db.create_all()`, which creates the `Startup` table if it does not already exist. The call is wrapped in a `try` block that writes a warning to the log instead of raising an error. This is deliberate: if the database is slow or still waking up, the web process must still start and serve the pages that do not need the database.

`init_db()` is called at module level, outside the `if __name__ == "__main__"` block. Gunicorn imports `app:app` and never runs the main block, so putting the call at module level makes sure the table is created both when the developer runs `python app.py` and when the application runs in production. The main block simply calls `app.run(debug=True)`, which starts Flask's development server with automatic reloading and detailed error pages for local use.

5.15 Validation, Error Handling and User Feedback

The website validates data at several levels. At the file level, only the five allowed image types are accepted and the total request size is limited to 8 MB. At the field level, `request.form["name"]` raises a `KeyError` if a required field is missing, and `parse_date` and `parse_time` raise a `ValueError` if a value is in the wrong format. Both exceptions are caught in one `except (KeyError, ValueError)` block, which shows the message "Could not save record: please check the form fields" together with the cause. At the page level, a missing report file produces a 404 error, and an unreachable database produces a 503 response from the health route.

User feedback is given through `flash()`. A message is stored in the session in one request and displayed once by the template in the next request, after the redirect. Success messages are

tagged `success` and errors are tagged `error`, so the template can show them in different colours.

5.16 Security and Reliability Considerations

- **Secrets from the environment.** The secret key, the database settings and the Cloudinary address are read from environment variables, so they can be set in the hosting dashboard. Good practice, and a planned improvement, is to remove every fallback value from the source code so that no credential is stored in it.
- **Encrypted database traffic.** The connection to the managed MySQL service uses TLS.
- **Safe file handling.** Extensions are checked, `secure_filename()` cleans local file names, existing files are never overwritten and the request size is limited.
- **Template escaping.** Jinja2 escapes text by default, so text typed into the form cannot inject HTML into the pages.
- **Resilient connections.** Pre-ping and recycling prevent errors caused by connections closed by the server.
- **Known limitation.** The Field Work form has no login, so anyone who knows the address can add a record. This is acceptable for a college project with a single team, but a login page is listed under future scope in Chapter 6.

5.17 Deployment on Render

The website is deployed on Render as a web service connected to the project repository. Render installs the dependencies, starts the application with Gunicorn using `app:app`, and provides a public HTTPS address. The settings that differ between the developer's computer and the live website, such as the database details and the Cloudinary address, are entered as environment variables in the Render dashboard, so that the same code runs in both places. Each time the repository is updated, Render rebuilds and restarts the service. Because the local disk is erased at every restart, the photographs are kept in Cloudinary and the records are kept in the managed MySQL service, and neither is affected by a redeployment.

5.18 Environment Variables Used by the Application

Table 5.5: Environment variables read by app.py

Variable	Purpose
`SECRET_KEY`	Signs the session cookie used for flash messages

Variable	Purpose
DB_USER, DB_PASSWORD, DB_HOST, DB_PORT, DB_NAME	Connection details of the MySQL database
DB_SSL_MODE	Set to DISABLED to turn off TLS for a local database; encryption is on by default
CLOUDINARY_URL	Enables Cloudinary photo storage and carries its credentials
DEMO_VIDEO_CROP_URL, DEMO_VIDEO_BG_URL, DEMO_VIDEO_RENAMER_URL	Optional external addresses of the three demo videos
DEMO_SITE_IMAGE_URL	Optional external address of the class management website screenshot
KEEP_ALIVE_URL, RENDER_EXTERNAL_URL, KEEP_ALIVE	Control the background keep-alive thread

5.19 Functional Verification

The table lists the checks that can be used to verify each feature of the website and the behaviour expected from each one.

Table 5.6: Functional checks and expected behaviour

Feature	Action	Expected behaviour
Navigation	Click Home, Dashboard, Field Work, Demos and Report	Each page opens and the current item is highlighted
Add a visit	Fill the form and press save	A success message appears and the new card is shown at the top of the list
Photo upload	Attach a .jpg file	The photograph appears on the card and still appears after the service restarts
Wrong file type	Attach a .pdf as the photograph	The photograph is ignored and the record is saved without it
Wrong date or time	Submit an invalid value	An error message is shown and no record is created
Dashboard totals	Add a visit in a new village	The place count and the visit count increase
Report page	Open /report	The PDF opens in the viewer; Download PDF saves the file

Feature	Action	Expected behaviour
Missing report	Remove the PDF from static	Report routes return a 404 page
Health check	Open /healthz	The text "ok" is returned

5.20 Limitations and Planned Improvements

The website meets the purpose for which it was built, but several improvements are possible. Records can be added but not yet edited or deleted from the website. The photograph is uploaded before the other form fields are validated, so a rejected form can leave an unused image in Cloudinary. There is no search or filter on the dashboard, and the pages are in English only. These points are listed in Section 6.6 as future scope and would be natural tasks for a later batch of students.

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The project showed that direct, hands-on technical support, combined with purpose-built software tools and a record of outcomes, is an effective way to promote digital adoption among rural micro-enterprises. Unlike one-off workshops or pamphlets, working alongside each entrepreneur to set up tools and solve specific problems produced real adoption.

The project also showed that training alone is not enough. Follow-up visits were essential to sustain new habits, and the software tools addressed problems that training could not solve. The project website serves as a record of the field work and as a resource for future cohorts.

Across the four enterprises, the most consistent lesson was that the founders already knew what they needed. They were waiting for someone to sit with them, on their own device, and help them get the tool working. That is the specific contribution a field visit can make.

The software component supported this work in a lasting way. The project website turned scattered notes, photographs and videos into a structured record that can be searched, shown to examiners and extended by later batches. Its design, from the routes that serve each page to the upload workflow that stores each photograph, is described in Chapter 5.

6.2 Key Findings

- Hands-on support overcomes hesitation: entrepreneurs who had never used digital tools adopted them when the setup was done with them, not for them.
- Follow-up is essential: habits introduced on the first day rarely lasted the week without a second check-in.
- Fit beats sophistication: tools that matched existing record-keeping habits were adopted faster than those that replaced them.
- Purpose-built tools fill gaps: the auto-crop, background removal and camera renamer tools solved problems that training alone could not.
- Documentation matters: storing visit records in a database and presenting them on a website created a record that outlasts any single visit.
- A local cluster exists: three of the four enterprises faced the same photo-processing problem, which suggests wider demand for these tools.

6.3 Significance of Findings

The findings suggest that rural digital adoption is not mainly a knowledge problem but a support and fit problem. Most entrepreneurs understand why digital tools are useful. What they need is someone to set things up with them, and simple tools that work within their existing habits. Colleges running community engagement programmes are well placed to provide this kind of sustained, hands-on support.

6.4 Recommendations

- Continue the three-visit model of needs assessment, hands-on setup and follow-up review, and do not skip the follow-up.
- Build for fit, not features: start tool development from observed problems, not from assumptions.
- Document every visit, including founder statements and before and after status.
- Make the project records available so that future cohorts can build on the existing work.
- Set up tools on the entrepreneur's own device and accounts, not on a demo device.
- When several enterprises face the same problem, build one reusable tool instead of solving each case separately.

6.5 Challenges Faced

The project met several practical difficulties, and the way each was handled is part of what was learned.

- **Limited connectivity:** rural enterprises cannot always rely on fast internet. This is why the photo tools were built to work locally and why the website was kept light.
- **Non-persistent hosting storage:** the free hosting plan erases local files whenever the application restarts, so photographs uploaded to the server disappeared. The problem was solved by moving photo storage to Cloudinary and storing only the permanent link in the database.
- **Sleeping server:** the free plan stops an inactive web service, so the first visitor waited a long time. A health-check route and a background keep-alive thread were added to reduce this delay.
- **Database connections dropping:** a quiet database connection could be closed by the server, causing the next request to fail. Connection pre-checking and recycling were enabled in the database settings to prevent this.

- **Variety of photographs:** school photographs differ in lighting, distance and posture, so the cropping tool had to be checked on different kinds of photographs to give consistent framing.
- **Limited time:** the visits, the development and the report had to be completed within one semester, which limited the length of the follow-up period.

6.6 Future Scope

- Add a login page so that only the project team can add or edit field records, while the dashboard and the report remain public.
- Add editing, deleting and searching of records, and an option to export the records as a spreadsheet.
- Move all passwords and keys out of the source code and into environment variables only, and rotate them regularly.
- Package the three photo tools into a single desktop application with a simple graphical window, so that a school photographer can run a full batch with one click.
- Add support for Marathi and Hindi on the website, because many of the entrepreneurs prefer to read in their own language.
- Track adoption over a longer period, for example three and six months after each visit, and record the results in the same database.
- Extend the programme to other kinds of enterprises, such as tailoring shops and small food businesses, and compare the problems found.

6.7 Final Remark

The project at B.N.N. College, Bhiwandi, has shown that meaningful digital adoption in rural areas is achievable when support is direct, patient and matched to the entrepreneur’s actual habits. The field visits, the software tools and the project website together form a record of what was done and a foundation for what can be done next.

CHAPTER 7: REFERENCES

The following references are relevant to the subject area of the project:

[1] Ministry of Electronics and Information Technology, Government of India (2023). Digital Personal Data Protection Act, 2023.
Official MeitY Page: Data Protection Framework

[2] National Payments Corporation of India (2024). UPI Product Statistics.
Official NPCI Page: UPI Product Statistics

[3] Google (2025). Google Business Profile: Official Documentation.
Google Business Profile Help

[4] WhatsApp Business (2025). WhatsApp Business Catalogue: Product Guide.
WhatsApp Business: Catalogue

[5] Telecom Regulatory Authority of India (2024). Annual Report on Internet Penetration in Rural India.
Official TRAI Website

[6] NASSCOM (2023). Digital Adoption in Indian Micro-Enterprises.
Official NASSCOM Website

[7] Project Website: Technical Support for Rural Startups.
Project Website

[8] Pallets Projects (2025). Flask Documentation.
Flask Documentation

[9] Pallets-Eco (2025). Flask-SQLAlchemy Documentation.
Flask-SQLAlchemy Documentation

[10] SQLAlchemy (2025). SQLAlchemy Documentation.
SQLAlchemy Documentation

[11] PyMySQL (2025). PyMySQL Documentation.
PyMySQL Documentation

[12] Cloudinary (2025). Python SDK Integration Guide.
Cloudinary Python Integration

[13] Render (2025). Render Documentation: Web Services.
Render Web Services

[14] Gunicorn (2025). Gunicorn Documentation.
Gunicorn Documentation

[15] Deng, J., Guo, J., Ververas, E., Kotsia, I. and Zafeiriou, S. (2020). RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild. CVPR 2020.
RetinaFace Paper (arXiv)

[16] Pillow (2025). Pillow Documentation.
Pillow Documentation

[17] Aiven (2025). Aiven for MySQL.
Aiven for MySQL

[18] Tailwind Labs (2025). Tailwind CSS Documentation.
Tailwind CSS Documentation